🌐 Projecte: Servidor de Chat Multi-Client en Java

Resultat d'Aprenentatge: Desenvolupa aplicacions que ofereixen serveis en xarxa, utilitzant llibreries de classes i aplicant criteris d'eficiència i disponibilitat.

1. Estructura del Projecte

```
ChatServidor/
├── src/
│   ├── servidor/
│   │   ├── ServidorChat.java      // Servidor principal
│   │   └── GestorClient.java      // Gestiona cada client en un fil
│   └── client/
│       ├── ClientChat.java               // Client de chat
│       └── VerificadorDisponibilitat.java// Comprova disponibilitat
└── README.md
```

2. Conceptes que s'apliquen

2.1 – Protocols estàndard de comunicació (C.1.1)

Aquest projecte utilitza el protocol **TCP/IP**:

- TCP (Transmission Control Protocol): garanteix que els paquets arriben en ordre i sense pèrdues.
- IP (Internet Protocol): s'ocupa de l'encaminament dels paquets per la xarxa.

Per a un chat, **TCP** és la millor opció perquè no volem perdre cap missatge. Si volguéssim una aplicació on la pèrdua puntual d'una dada no importés (per exemple, una càmera en temps real), usaríem **UDP**.

2.2 – Avantatges dels protocols estàndard (C.1.2)

- Interoperabilitat: qualsevol client (fins i tot en un altre llenguatge) es pot connectar al nostre servidor.
- Fiabilitat: TCP ens evita implementar mecanismes de reintent manualment.
- Compatibilitat: els Sockets de Java segueixen l'estàndard BSD Sockets, usat arreu.

2.3 – Llibreries de Java emprades (C.1.3)

Classe	Paquet	Funció
ServerSocket	java.net	Escolta connexions entrants
Socket	java.net	Representa una connexió TCP
BufferedReader	java.io	Llegeix text per línies eficientment

PrintWriter	java.io	Escriu text amb autoFlush
Thread	java.lang	Permet executar codi en paral·lel

2.4 – Servei de comunicació en xarxa (C.1.4)

El servidor implementa el flux següent:

```
Servidor arrenca  → obre ServerSocket al port 12345
    └── Arriba client → accept() retorna un Socket
            └── Es crea un GestorClient i s'arrenca en un Thread
                    └── El gestor llegeix/escriu fins a "sortir"
```

2.5 – Clients de verificació (C.1.5)

A part del **ClientChat**, tenim el **VerificadorDisponibilitat** que és una eina de diagnòstic per comprovar si el servidor respon.

2.6 – Comunicació simultània de diversos clients (C.1.6)

Cada client s'atén en un **fil independent (Thread)**:

```
// A ServidorChat.java
GestorClient gestor = new GestorClient(socketClient);
new Thread(gestor).start();  // ← cada client = fil nou
```

Sense fils, el servidor hauria d'esperar que un client acabés per atendre el següent. Amb fils, tots es comuniquen alhora.

La paraula clau **synchronized** als mètodes **broadcast** i **eliminarClient** evita condicions de carrera (race conditions) quan dos fils modifiquen la llista alhora.

2.7 – Verificació de disponibilitat (C.1.7)

VerificadorDisponibilitat.java intenta obrir un **Socket** amb un timeout de 2 segons:

- Connexió OK → servei disponible
- IOException → servei no disponible

2.8 – Depuració i documentació (C.1.8)

- Tots els fitxers estan documentats amb comentaris.
- El servidor imprimeix per consola cada connexió/desconnexió.
- Els errors es mostren amb missatges descriptius per System.err.

3. Com executar el projecte

Pas 1: Compilar

Des de la carpeta **src/**:

```
# Des de la carpeta src/
javac servidor/ServidorChat.java servidor/GestorClient.java
javac client/ClientChat.java client/VerificadorDisponibilitat.java
```

Pas 2: Arrencar el servidor

```
java servidor.ServidorChat
```

Hauries de veure:

```
=== SERVIDOR DE CHAT INICIANT ===
Escoltant al port: 12345
Esperant clients...
```

Pas 3: Verificar que el servidor funciona

En un **nou terminal**:

```
java client.VerificadorDisponibilitat
```

Resultat esperat:

```
✓ SERVEI DISPONIBLE - El servidor de chat està en marxa.
```

Pas 4: Connectar clients

Obre **dos o més terminals** i executa en cada un:

```
java client.ClientChat
```

Introdueix un nom d'usuari quan et demani i comença a escriure!

Per sortir escriu:

```
sortir
```

4. Exemple de sessió

Terminal Servidor:

```
=== SERVIDOR DE CHAT INICIANT ===
Escoltant al port: 12345
[+] Nou client connectat des de: 127.0.0.1
[+] L'usuari 'Anna' s'ha unit al chat.
[+] Nou client connectat des de: 127.0.0.1
[+] L'usuari 'Bernat' s'ha unit al chat.
[Anna]: Hola Bernat!
[-] L'usuari 'Bernat' s'ha desconnectat.
```

Terminal Client (Anna):

```
Benvingut/da al chat! Escriu el teu nom:
Anna
Connectat/da! Ja pots escriure missatges.
*** Bernat s'ha unit al chat ***
Hola Bernat!
[Bernat]: Hola Anna!
*** Bernat ha sortit del chat ***
```

5. Preguntes de comprensió

1. Per què usem synchronized als mètodes broadcast i eliminarClient?
2. Quin seria el problema si NO usem fils (Threads) per a cada client?
3. Per quina raó BufferedReader és més eficient que InputStreamReader sol?
4. Quin protocol de capa de transport estem usant? TCP o UDP? Per què?
5. Modifica el projecte perquè el servidor registri en un fitxer .log tots els missatges.

6. Possibles millores (exercicis opcionals)

- Missatges privats: implementar /privat NomUsuari missatge
- Llista d'usuaris: comanda /usuaris que mostri qui hi ha connectat
- Historial: guardar els últims N missatges i enviar-los a qui es connecta
- Interfície gràfica: afegir una GUI amb Swing o JavaFX al client
- Xifrat: usar SSLSocket en lloc de Socket per connexions segures